Moving Metrics, Not Data: Low-Cost Ingestion From Heterogeneous Sources for Proactive Business Intelligence

Palla Chetana Reddy, Varshini Yadav Nasaram, Obulagari Nandini

School of Technology, Woxsen University

Capstone Project-I

[Instructor name]

October 7, 2026

Abstract

Business intelligence systems that monitor metrics on their own must first collect data from systems that differ from company to company, and this step is usually the most expensive one. Managed extract-load tools charge by the number of rows copied, self-hosted platforms are heavy to operate, and bulk file transfer over a single FTP connection is slow for large exports. This paper describes the ingestion layer of KAIROS, an autonomous business intelligence agent, which is built on one principle: move computed metrics rather than raw data. Five protocol-level adapters (SQL, SFTP/FTP, files and object storage, REST, and push) replace per-product connectors; incremental watermarks and idempotent upserts make every sync repeatable; aggregation runs inside the source database or in an edge agent next to large files; and a chunked, compressed, parallel and resumable transfer service handles the cases in which a raw file must move. In a live demonstration, a 100,305-row ERP table was synchronised by sending 19.5 KB, and only one new file was downloaded from an FTP server holding fifteen. For a 105.6 MB export over links capped at 50 Mbit/s, the transfer service was 6.4 times faster than single-stream FTP, and edge aggregation sent 193 KB in 1.3 s.

Keywords: data ingestion, ELT, FTP, edge computing, file transfer, business intelligence, schema matching

Moving Metrics, Not Data: Low-Cost Ingestion From Heterogeneous Sources for Proactive Business Intelligence

A business intelligence (BI) agent that watches metrics without being asked is only as useful as the data it can reach. In a typical mid-sized company, orders sit in an ERP database, marketing exports land in a shared folder every night, a logistics partner drops compressed files on an FTP server, and the support platform offers a paged REST API. Connecting all of these is often the costliest part of a BI project. Managed extract-load-transform (ELT) services such as Fivetran charge per monthly active row (Fivetran, n.d.), open-source platforms such as Airbyte need several containers per connection (Airbyte, n.d.), and each company combines different products, so a catalogue of per-product connectors never covers everyone.

This question was raised directly during the mid-term review of KAIROS, an autonomous causal BI agent built as a capstone project. The reviewers asked how data could come from any source when connectors are heavy and expensive, whether FTP could be used, and how large files would be handled, since FTP is slow for them. They also mentioned a tool built by another student that transfers large files directly. This paper answers those questions with the design and measurements of the KAIROS ingestion layer.

The central observation is that a monitoring system needs metrics, not records. Fifty key performance indicators at 15-minute resolution amount to about 4,800 rows a day, whereas the underlying order table may grow by millions of rows. If the aggregation runs where the data lives, the transfer problem largely disappears, a principle long used in large-scale data processing (Dean & Ghemawat, 2008; Melnik et al., 2010). The rest of the design follows from this observation.

Related Work

Managed ELT services copy whole tables into a warehouse with maintained connectors (Fivetran, n.d.); they are reliable but priced by volume and lock the customer to a vendor. Airbyte offers a large open-source connector catalogue but is designed for container platforms (Airbyte, n.d.). Change data capture tools such as Debezium stream row-level changes from database logs (Debezium, n.d.), which is efficient for replication but still moves raw rows. Kleppmann (2017) describes idempotent writes and incremental processing as the basis of reliable data pipelines, and both are used here.

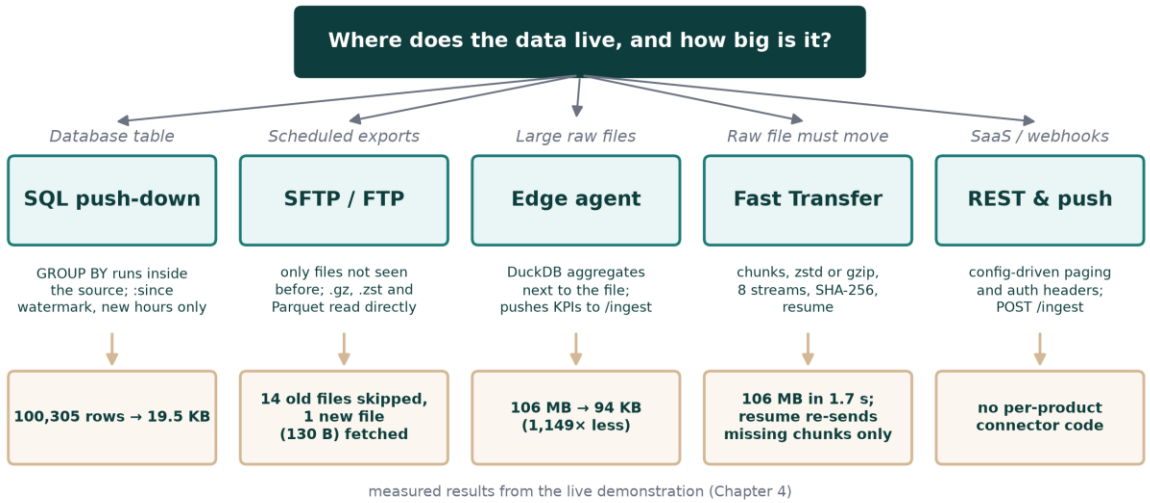
For large files, FTP (Postel & Reynolds, 1985) and its secure variants run over a single TCP connection whose throughput is bounded by window size and round-trip time (Mathis et al., 1997); opening several parallel connections raises throughput on long or lossy paths (Hacker et al., 2002). Commercial tools such as IBM Aspera combine their own transport protocol with resume and encryption (IBM, n.d.). Edge computing moves processing towards the data source to reduce bandwidth and latency (Shi et al., 2016). Automatic schema matching, as surveyed by Rahm and Bernstein (2001), reduces the manual effort of mapping new sources; KAIROS applies a lightweight version of it.

Design

Figure 1 shows how the ingestion layer routes a source according to where its data lives and how large it is. Every route ends in the same canonical form, rows of source, metric name, timestamp and value, which are upserted into an embedded DuckDB store (Raasveldt & Mühleisen, 2019) with that tuple as the primary key.

Figure 1

Ingestion Routes by Data Location and Size



Note. Bottom row: results measured in the live demonstration described under Evaluation.

Protocol Adapters Instead of Product Connectors

Almost every business system exposes data through a small number of protocols, so KAIROS implements protocols rather than products. A SQL adapter covers any database with a SQLAlchemy driver; a file adapter reads CSV, Excel, Parquet and JSON, compressed or not, from local paths, globs, S3 or HTTPS; an SFTP/FTP adapter fetches scheduled exports (Ylonen & Lonvick, 2006); a configurable REST adapter handles authentication headers, pagination and incremental parameters (Fielding, 2000); and a push endpoint accepts records from webhooks and from the edge agent. Every adapter implements one contract that returns canonical rows and an updated state. A new company is therefore added by configuration, with secrets read from environment variables.

Incremental, Repeatable Synchronisation

Each source keeps a small state between runs. Database and REST sources keep a timestamp watermark that is passed to the next query as a `:since` parameter. File and FTP

sources keep the name, size and modification time of every file already processed, so a sync downloads only files it has not seen; half-written files are skipped until they are older than a set age or a marker file appears. Because writes are upserts on the primary key, re-reading an overlapping window, which is necessary when the last hourly bucket was still filling, updates values instead of duplicating them (Kleppmann, 2017). Sources run in parallel with individual timeouts, so one slow system cannot delay the others.

Aggregation Push-Down and the Edge Agent

For database sources, the configured query performs the aggregation, for example summing order amounts by hour, so the database returns KPI rows instead of order rows. For large files, the file adapter asks DuckDB to aggregate while it streams the file, and a stand-alone edge agent does the same inside the company network before pushing only the resulting KPIs to KAIROS. Raw data never leaves the company in this route, which also helps with privacy.

Fast Transfer for Raw Files

Some cases still require the raw file, for example a historical backfill. The transfer service splits the file into chunks of 1 to 32 MB. Each chunk is compressed with gzip or Zstandard (Collet & Kucherawy, 2021), hashed with SHA-256 and uploaded over several parallel connections, with retries and exponential back-off for each chunk. The server checks the length and hash of every chunk, writes it at its offset in a pre-sized file and only then records it as received. After a dropped connection, the client asks for the list of missing chunks and sends only those. On completion, the server re-hashes the whole file and can ingest it immediately.

Semantic Mapping of New Sources

When a source is added, a Semantic Layer Auto-Construction (SLAC) step inspects a sample. It identifies the timestamp column, decides whether the data is in long form (one row per

metric and time) or wide form (one column per metric), excludes identifier columns such as order numbers, and detects irregular timestamps that indicate raw records needing aggregation. It then proposes a complete mapping that the user can accept or edit, in the spirit of automatic schema matching (Rahm & Bernstein, 2001).

Evaluation

Method

Four simulated company systems were set up on one machine: a SQLite ERP database of 100,305 orders, a folder of three weekly marketing CSV exports, an FTP server holding fourteen daily compressed logistics files, and a paged REST API. Their schemas differ, so each needed its own mapping. For the transfer benchmark, a raw export of two million orders (105.6 MB as CSV) was moved by each method between two separate processes, with every TCP connection capped at 50 Mbit/s to emulate a wide-area link. The cap was applied in the same way to FTP and to the transfer service, and byte counts are exact.

Results

Table 1 lists the steps of the live demonstration. The first synchronisation collected 4,704 KPI rows from the four systems, and the ERP route sent only 19.5 KB because the grouping ran inside the database. A second synchronisation moved no new rows from the file, FTP or REST sources. When a new file was placed on the FTP server, only that file was downloaded.

Table 1

Live Ingestion Demonstration

Step	Result
First sync of four sources	4,704 KPI rows; ERP sent 19.5 KB for 100,305 raw rows
Second sync	No new rows from FTP, files or REST; nothing duplicated
New file on the FTP server	Only that file (130 bytes) downloaded; 14 skipped
Edge agent on a 105.6 MB export	94.1 KB pushed (1,149 times less data)

Step	Result
Fast Transfer of the same file	28.7 MB sent in 1.7 s; SHA-256 verified
Connection cut after 13 of 27 chunks	Only the 14 missing chunks re-sent

Table 2 and Figure 2 compare the transfer methods. Compressing the file before a single FTP upload almost halved the time, but most of the gain was spent on compression. The parallel transfer service was 6.1 times faster than plain FTP with gzip and 6.4 times faster with Zstandard, sending about a quarter of the bytes. After a simulated failure halfway through, completing the transfer took 1.65 s, because only the missing half was sent. The edge agent was the fastest route of all and reduced traffic by a factor of about 550.

Table 2

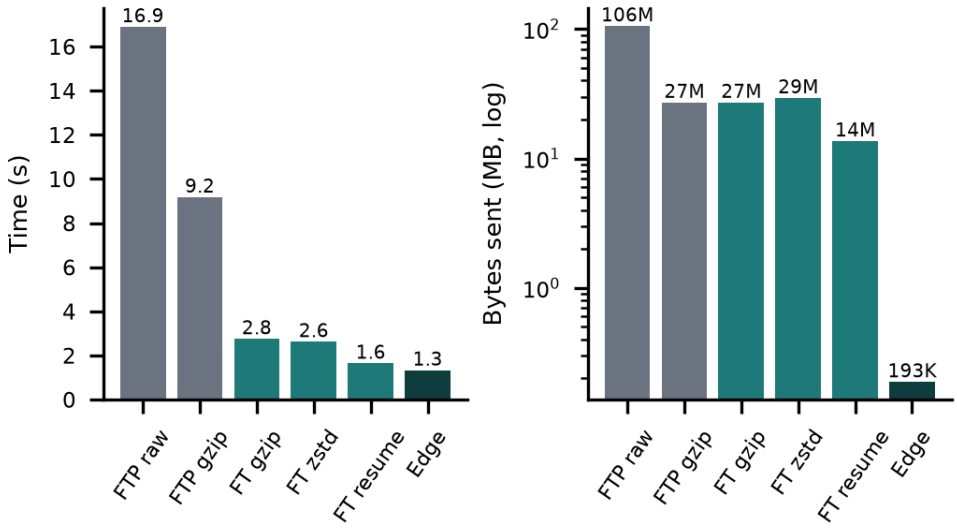
Transfer of a 105.6 MB Raw Export Over Links Capped at 50 Mbit/s

Method	Time (s)	Bytes sent	Speed-up
FTP, single stream, raw CSV	16.90	105.6 MB	1.0×
FTP, single stream, gzip	9.18	27.1 MB	1.8×
Fast Transfer, 8 streams, gzip	2.76	27.1 MB	6.1×
Fast Transfer, 8 streams, zstd	2.63	29.4 MB	6.4×
Fast Transfer, resumed after 50%	1.65	13.6 MB	10.2×
Edge agent (KPIs only)	1.34	192.7 KB	12.6×

Note. The gzip FTP time includes 4.8 s of compression. Speed-up is relative to single-stream FTP of the raw file.

Figure 2

Transfer Time and Bytes on the Wire for Each Method



Note. FT = Fast Transfer. The right panel uses a logarithmic scale.

Discussion

The measurements support a simple answer to the review questions. Data can come from almost any system through five protocol adapters, configured rather than coded, at no licence cost. FTP is a valid route, and it is fast when it carries small, compressed, new files only. Large data should usually not be transferred at all: aggregating in the source database or with an edge agent turned a 105.6 MB problem into a few hundred kilobytes. When a raw file is genuinely needed, chunking, compression, parallel connections and resume recover most of the gap to the link capacity, which is the same set of techniques used by commercial high-speed transfer tools (IBM, n.d.; Hacker et al., 2002).

The work has limitations. The benchmark ran on one machine with an emulated link, and real long-distance links with packet loss may show a different speed-up. The demonstration systems were simulated rather than taken from a partner company. Plain FTP was used in the

demonstration for convenience, although production use requires SFTP or FTPS with read-only accounts. Change data capture was not implemented, and the SLAC mapping relies on rules for English column names.

Conclusion

The cost and speed of BI ingestion depend mostly on how much data is moved rather than on the protocol used to move it. By moving computed metrics instead of raw records, and by providing a resumable parallel transfer for the remaining cases, KAIROS connects heterogeneous company systems at negligible cost. Future work will measure the transfer service over real wide-area links, add change data capture for databases, and evaluate the approach with a partner company.

References

Airbyte. (n.d.). *Airbyte documentation*. Retrieved October 6, 2026, from https://docs.airbyte.com/

Collet, Y., & Kucherawy, M. (2021). *Zstandard compression and the application/zstd media type* (RFC 8878). Internet Engineering Task Force. https://doi.org/10.17487/RFC8878

Dean, J., & Ghemawat, S. (2008). MapReduce: Simplified data processing on large clusters. *Communications of the ACM, 51*(1), 107–113. https://doi.org/10.1145/1327452.1327492

Debezium. (n.d.). *Debezium documentation*. Red Hat. Retrieved October 6, 2026, from https://debezium.io/documentation/

Fielding, R. T. (2000). *Architectural styles and the design of network-based software architectures* [Doctoral dissertation, University of California, Irvine].

Fivetran. (n.d.). *Usage-based pricing: Monthly active rows*. Retrieved October 6, 2026, from https://fivetran.com/docs/usage-based-pricing

Hacker, T. J., Athey, B. D., & Noble, B. (2002). The end-to-end performance effects of parallel TCP sockets on a lossy wide-area network. In *Proceedings of the 16th International Parallel and Distributed Processing Symposium*. IEEE.

IBM. (n.d.). *IBM Aspera*. Retrieved October 6, 2026, from https://www.ibm.com/products/aspera

Kleppmann, M. (2017). *Designing data-intensive applications*. O'Reilly Media.

Mathis, M., Semke, J., Mahdavi, J., & Ott, T. (1997). The macroscopic behavior of the TCP congestion avoidance algorithm. *ACM SIGCOMM Computer Communication Review, 27*(3), 67–82. https://doi.org/10.1145/263932.264023

Melnik, S., Gubarev, A., Long, J. J., Romer, G., Shivakumar, S., Tolton, M., & Vassilakis, T. (2010). Dremel: Interactive analysis of web-scale datasets. *Proceedings of the VLDB Endowment, 3*(1–2), 330–339. https://doi.org/10.14778/1920841.1920886

Postel, J., & Reynolds, J. (1985). *File Transfer Protocol (FTP)* (RFC 959). Internet Engineering Task Force. https://doi.org/10.17487/RFC0959

Raasveldt, M., & Mühleisen, H. (2019). DuckDB: An embeddable analytical database. In *Proceedings of the 2019 International Conference on Management of Data* (pp. 1981–1984). ACM. https://doi.org/10.1145/3299869.3320212

Rahm, E., & Bernstein, P. A. (2001). A survey of approaches to automatic schema matching. *The VLDB Journal, 10*(4), 334–350. https://doi.org/10.1007/s007780100057

Shi, W., Cao, J., Zhang, Q., Li, Y., & Xu, L. (2016). Edge computing: Vision and challenges. *IEEE Internet of Things Journal, 3*(5), 637–646. https://doi.org/10.1109/JIOT.2016.2579198

Ylonen, T., & Lonvick, C. (2006). *The Secure Shell (SSH) protocol architecture* (RFC 4251). Internet Engineering Task Force. https://doi.org/10.17487/RFC4251